

Module 3: Lesson 2

Deep Convolutional Neural Networks: Transfer Learning for CNNs



Outline

- ▶ Transfer Learning
- ▶ Xception
- ▶ Application: Stock return prediction with candlestick charts

Transfer Learning

Many CNN architectures have been proven successful at performing image recognition tasks.

- ▶ These architectures are usually large-scale models trained with large-scale datasets.

Transfer Learning: We can leverage those trained models and apply them to our, possibly smaller-scale, applications.

- ▶ The learning ability of the pre-trained model may capture general features of images, which may prove effective for recognizing the objects in our target dataset.

Steps in Transfer Learning:

1. Obtain the parameters of a pre-trained model from a source dataset.
2. Create the target model, copying the structure and parameter values of the pre-trained network, except the output layers.
3. Add output layers to the target model, adapted to the labels of our target database.
4. Train the target model on the target dataset. The pre-trained model parameters can be fine-tuned or kept fixed.

Xception: Separable convolutions

Xception is a CNN architecture that has been proposed as a generalization of Inception.

The architecture replaces the Inception blocks by layers of “separable convolutions.”

Separable convolutions involve two steps:

1. Apply a convolution operator to each input channel using a single kernel.
2. Apply 1×1 convolutions to the tensors that result from the previous steps. The number of 1×1 kernels, with a dimension equal to the number of input channels, must equal the number of output channels.

This operator reduces considerably the number of operations that are involved in a standard convolution operator.

The intuition follows VGG models: Deep networks are preferred over wide networks. Separable convolutions reduce the computational costs of increasing the depth of the architecture.

Xception: Example of separable convolution (First step)

INPUT		KERNEL		OUTPUT																	
<table><tr><td>1</td><td>2</td><td>4</td></tr><tr><td>3</td><td>7</td><td>2</td></tr><tr><td>2</td><td>0</td><td>2</td></tr></table>	1	2	4	3	7	2	2	0	2	*	<table><tr><td>3</td><td>2</td></tr><tr><td>2</td><td>1</td></tr></table>	3	2	2	1	=	<table><tr><td>20</td><td>30</td></tr><tr><td>27</td><td>27</td></tr></table>	20	30	27	27
1	2	4																			
3	7	2																			
2	0	2																			
3	2																				
2	1																				
20	30																				
27	27																				
<table><tr><td>0</td><td>2</td><td>1</td></tr><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>2</td><td>4</td><td>0</td></tr></table>	0	2	1	1	2	3	2	4	0	*	<table><tr><td>1</td><td>2</td></tr><tr><td>0</td><td>3</td></tr></table>	1	2	0	3	=	<table><tr><td>10</td><td>13</td></tr><tr><td>17</td><td>8</td></tr></table>	10	13	17	8
0	2	1																			
1	2	3																			
2	4	0																			
1	2																				
0	3																				
10	13																				
17	8																				
<table><tr><td>4</td><td>2</td><td>1</td></tr><tr><td>5</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr></table>	4	2	1	5	1	0	1	0	1	*	<table><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>2</td></tr></table>	0	1	1	2	=	<table><tr><td>9</td><td>2</td></tr><tr><td>2</td><td>2</td></tr></table>	9	2	2	2
4	2	1																			
5	1	0																			
1	0	1																			
0	1																				
1	2																				
9	2																				
2	2																				

Remember that in the standard convolution operator we would have as many kernel elements as output channels, each with dimension kernel height x kernel width x input channels.

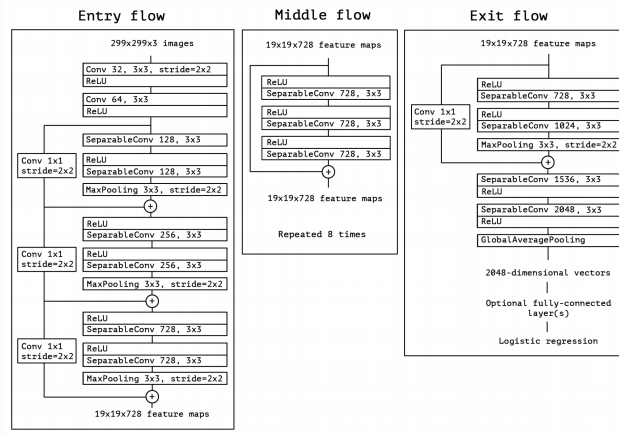
- In separable convolutions, we have just a single kernel element in the first step of computations.

Xception: Example of separable convolution (Second step)

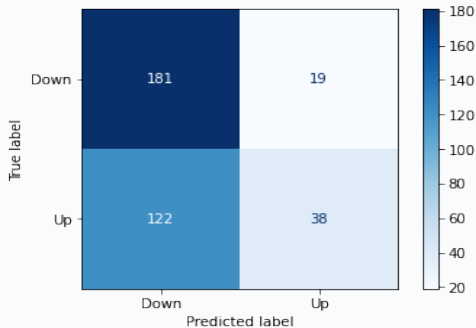
INPUT		KERNEL		OUTPUT									
<table border="1"><tr><td>20</td><td>30</td></tr><tr><td>27</td><td>27</td></tr></table>	20	30	27	27	*	<table border="1"><tr><td>-1</td></tr></table>	-1						
20	30												
27	27												
-1													
		+											
<table border="1"><tr><td>10</td><td>13</td></tr><tr><td>17</td><td>8</td></tr></table>	10	13	17	8	*	<table border="1"><tr><td>1</td></tr></table>	1	=	<table border="1"><tr><td>8</td><td>-13</td></tr><tr><td>-6</td><td>-15</td></tr></table>	8	-13	-6	-15
10	13												
17	8												
1													
8	-13												
-6	-15												
		+											
<table border="1"><tr><td>9</td><td>2</td></tr><tr><td>2</td><td>2</td></tr></table>	9	2	2	2	*	<table border="1"><tr><td>2</td></tr></table>	2						
9	2												
2	2												
2													

We repeat the above operations with as many 1x1 kernels as output channels we have in the layer.

Xception: Architecture



Application: Stock return prediction with candlestick charts



We train the candlestick-based stock return prediction model using transfer learning and the Xception architecture.

After training the model several times, using transfer learning leads to better performance than the basic CNN architecture of Module 2.

However, the model still seems to predict too often a "down" move and displays overfitting.

Summary of Lesson 2

In Lesson 2, we have:

- ▶ Described transfer learning as a useful tool to train CNNs
- ▶ Described the Xception CNN architecture
- ▶ Applied transfer learning and Xception to predict stock returns using candlestick charts

⇒ **References for this lesson:**

Chapter 8 and Section 14.2 in [Zhang, Aston, et al. *Dive into Deep Learning*. 2022.](#)

TO DO NEXT: Please, go to the Jupyter Notebook associated with Lesson 2.

In the next lesson, we'll describe additional tools to enhance the performance of CNN architectures.